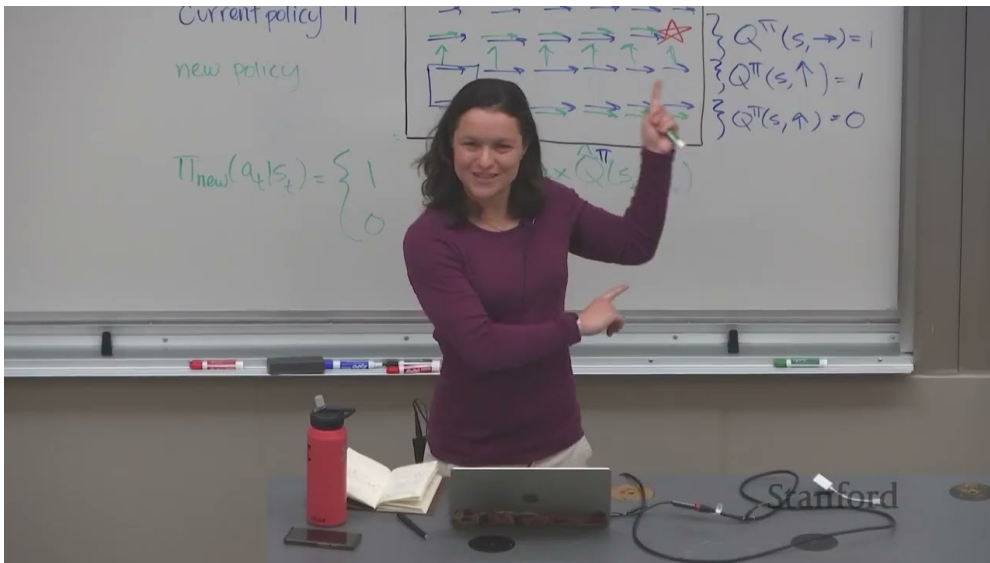


课程笔记

CS224R Lecture 6: Q-Learning

基于公开课程资料整理

2025 年春季



视频作者/频道: Stanford Online

发布日期: 2025-04

视频时长: 约 80 分钟

讲者: Chelsea Finn

课程主页: cs224r.stanford.edu

项目主页: <https://github.com/hqhq1025/ai-course-notes>

目录

1 从 Actor-Critic 到纯 Value-Based 方法	3
1.1 本章小结	3
2 Q-Learning 基础	3
2.1 最优 Q 函数	3
2.2 DQN 算法	3
2.3 本章小结	3
3 数据采集与 Replay Buffer 管理	4
3.1 好数据塑造通用性	4
3.2 优先经验回放与分布修正	4
3.3 案例：Atari Replay 抽样流程	4
3.4 本章小结	5
4 连续动作空间的挑战	5
4.1 本章小结	5
5 稳定化与正则化策略	6
5.1 Bootstrapping 与梯度归一	6
5.2 奖励塑形与行为规范	6
5.3 动作约束与输出裁剪	6
5.4 本章小结	6
6 Double Q-Learning 与过估计问题	6
6.1 过估计偏差	6
6.2 Double DQN	7
6.3 本章小结	7
7 评估、治理与部署	7
7.1 多层评估矩阵	7
7.2 部署与治理 checklist	7
7.3 Evidence Matrix 与操作时间线	8
7.4 Evidence Matrix 与操作时间线	8
7.5 本章小结	8
8 工程化视角：Q-Learning 工作流	9
8.1 数据 → 训练 → 部署闭环	9
8.2 部署监控策略	9
8.3 本章小结	9
9 可视证据与研究时间线	9
9.1 Slide 3: Q-Learning 模型管线	9
9.2 Slide 4: Temporal difference 与 replay dynamics	11
9.3 Slide 5: Deployment 监控	12

9.4	Slide 6: Action timeline and evidence matrix	13
9.5	Slide 7: Research directions	14
10	Research Directions & Open Questions	15
10.1	Scalable replay and prioritization	15
10.2	Robust evaluation and failure diagnostics	15
10.3	Agent orchestration and governance	15
10.4	本章小结	16
10.5	Sim-to-real 数据迁移	17
11	总结与延伸	17
11.1	核心总结表	17
11.2	进一步阅读	17
11.3	进一步阅读	17
11.4	本章小结	18
11.5	行动提醒	18

1 从 Actor-Critic 到纯 Value-Based 方法

前几讲介绍了 actor-critic 方法：同时维护策略（actor）和价值函数（critic）。Q-Learning 走得更远——完全去掉显式策略，只学习 Q 函数，然后从 Q 函数隐式地派生策略。

Value-Based vs. Actor-Critic

- **Actor-Critic**: 同时学习策略 π_θ 和价值函数 $\hat{V}$ 或 $\hat{Q}$ 。
- **Value-Based (Q-Learning)**: 只学习最优 Q 函数 Q^* ，策略隐式为 $\pi(s) = \arg \max_a Q^*(s, a)$ 。

1.1 本章小结

Q-Learning 是一种纯 value-based 方法，不需要显式的策略网络。

2 Q-Learning 基础

2.1 最优 Q 函数

Bellman 最优方程

最优 Q 函数 Q^* 满足：

$$Q^*(s, a) = r(s, a) + \gamma \max_{a'} Q^*(s', a')$$

关键性质： Q^* 对应的策略 $\pi^*(s) = \arg \max_a Q^*(s, a)$ 是所有策略中最优的。

与 actor-critic 中拟合 Q^π （当前策略的 Q 函数）不同，Q-Learning 直接拟合 Q^* （最优策略的 Q 函数），使用 $\max$ 而不是 $\mathbb{E}_{a' \sim \pi}$ 。

2.2 DQN 算法

Deep Q-Network (DQN) 使用深度神经网络来参数化 $Q_\phi(s, a)$ ：

$$\min_{\phi} \sum_{(s, a, r, s') \in \mathcal{B}} \left\| Q_\phi(s, a) - \left(r + \gamma \max_{a'} Q_{\phi'}(s', a') \right) \right\|^2$$

DQN 的关键技巧

- **Replay Buffer**: 存储所有历史转移，随机采样来打破数据相关性。
- **Target Network**: 使用延迟更新的 $Q_{\phi'}$ 计算 TD target，避免”自己追自己”的不稳定。
- **ϵ -Greedy 探索**: 以概率 ϵ 随机选择动作，否则选 $\arg \max_a Q(s, a)$ 。

2.3 本章小结

Q-Learning 通过拟合最优 Q 函数来间接获得最优策略。DQN 将 Q-Learning 扩展到高维状态空间，replay buffer 和 target network 是关键的稳定化技巧。

3 数据采集与 Replay Buffer 管理

3.1 好数据塑造通用性

Q-Learning 是典型的 off-policy 方法，训练过程中可重用历史转移，因此数据质量直接决定效果。

数据的三维质量框架

1. **Coverage**: 状态-动作对是否遍布整个训练分布。
2. **Freshness**: 旧数据可能已经过时，尤其当策略迭代很快时。
3. **Signal-to-Noise**: 奖励与下一状态之间的因果链是否足够清晰。

Q-Learning

CS 224R

1

图 1: Slide: Replay buffer 抽样的分布偏移与纠正策略

3.2 优先经验回放与分布修正

优先经验回放 (PER) 根据 TD Error 分配采样概率，使训练更多聚焦于高价值/高误差样本。修正方法包括：

- importance sampling weight 用以抵消非均匀采样
- alpha、beta 超参数控制采样与修正的强度

Replay Buffer 的工程实践提示

1. 定期清理过旧元素以防止“冷启动”偏移。
2. 保持 buffer size 在百万级别以平衡样本多样性。
3. 为不同任务维护多个 buffer (exploration vs. demonstration)。

3.3 案例：Atari Replay 抽样流程

以经典 Atari Q-Learning 为例，训练流水线可简化为以下步骤：

1. 通过 epsilon-greedy 收集原始 transition, 插入 'online buffer'。
2. 每 1k steps 用 prioritized sampling 生成 mini-batch, 同时记录 TD error distribution。
3. 将 TD error 作曲线可视化, 若长尾方向持续偏移, 触发 'replay augmentation' (32, 64 sampling)。
4. 定期将 high-variance batch 提前放入 'high-priority buffer' 供评估与 debugging。

Replay 监控 checklist

- 记录 replay hit ratio, 确保采样重用率在 70-95
- 'TD error' 上升时自动增加 'beta' 以增强 importance sampling 校正。
- 将 human-verified failure transitions 复制到 separate buffer, 方便 later inspection。

3.4 本章小结

Q-Learning 的 off-policy 特性让 replay buffer 成为性能发动机, 采样策略和数据质量直接决定最终泛化能力。

4 连续动作空间的挑战

DQN 原始形式只适用于**离散**动作空间 (网络输出每个动作的 Q 值, 然后取 $\arg\max$)。对于连续动作空间, $\max_a Q(s, a)$ 本身就是一个优化问题。

解决方案包括:

1. **离散化**: 将连续空间离散化, 但维度诅咒严重。
2. **采样**: 随机采样多个动作, 取 Q 值最大的。
3. **学习一个 actor**: 回到 actor-critic 框架 (如 SAC)。
4. **DDPG**: 训练一个确定性策略网络 $\mu_\theta(s) = \arg\max_a Q(s, a)$ 。

Q-Learning 在连续空间的困难

纯 Q-Learning 在连续动作空间中需要解决内层优化问题 $\max_a Q(s, a)$, 这在高维空间中计算代价很大。这也是为什么连续控制任务中 actor-critic 方法 (如 SAC) 通常比纯 Q-learning 更受欢迎。

4.1 本章小结

Q-Learning 在离散动作空间中自然高效, 但在连续动作空间中需要额外的技巧或回归到 actor-critic 方法。

5 稳定化与正则化策略

5.1 Bootstrapping 与梯度归一

Bootstrapping 将网络输出与自身目标绑定，容易引发梯度爆炸。实践中常用梯度裁剪和归一化来控制。

稳定技巧速查

- 梯度裁剪到 1.0，避免更新过大导致发散。
- Layer normalization 和 weight decay 控制输出范围。
- Double Q 的动作选择与估值拆分也属于稳定策略。

5.2 奖励塑形与行为规范

在高维任务中，直接使用 reward 会产生高方差。可以添加 shaping term 或 entropy penalty 平滑 learning signal。

奖励塑形的陷阱

shaping term 必须是 potential-based，否则可能改变最优策略；添加 penalty 也不能让 Q 值无限增大。

5.3 动作约束与输出裁剪

为了避免 Q 函数输出漂移，可采用 clipping 与 output constraint：

- 将 TD target 限制在 observed reward range $\pm$ margin，防止估值爆炸。
- 引入 Q regularization（如 L2 shrinkage toward previous target）使更新更平滑。
- 对 action 采样加噪后 clamp，保证 $\arg \max_a Q(s, a)$ 不会落入未见区域。

5.4 本章小结

稳定化需要同时从梯度信号和行为信号两个方向着手，避免值函数爆炸与策略漂移。

6 Double Q-Learning 与过估计问题

6.1 过估计偏差

$$\max_{a'} Q_{\phi}(s', a') \geq Q^*(s', \pi^*(s'))$$

取 max 操作会系统性地高估 Q 值，因为它选择的是噪声最大的方向。

6.2 Double DQN

使用当前网络选择动作，但用 target 网络评估：

$$y = r + \gamma Q_{\phi'} \left(s', \arg \max_{a'} Q_{\phi}(s', a') \right)$$

这在一定程度上缓解了过估计问题。

6.3 本章小结

Q-Learning 有系统性的过估计偏差。Double Q-Learning 通过分离动作选择和动作评估来缓解这一问题。

7 评估、治理与部署

7.1 多层评估矩阵

一个完整的评估流程应包含自动 benchmark、trajectory review 和 stress test：

- ‘score’、‘TD error’、‘episode length’ 等数值指标
- 手动审阅 failure trajectory / reward hack
- 噪声注入、belief shift、环境 perturbation 测试稳定性

层级	指标	触发策略
Training eval	Q target, loss, entropy	Divergence or reward plateau
Safety audit	reward hack detection, adversarial action	anomalous trajectories
Deployment sim	latency, repeatability, replicates	hardware change / rollout

表 1: 评估与治理矩阵

日志监控建议

记录 TD error distribution、max Q target、exploration ratio；设置 `td_error > threshold` 触发 replay augmentation, `reward variance > 2` 触发 reward shaping review。

7.2 部署与治理 checklist

上线前的三道“门”

1. Replay buffer 只能包含经过 human-in-the-loop 校验的 transitions。
2. Checkpoint 避免 reward hack，如 reward clipping + sanity check trajectories。
3. 设定 clear rollback 条件 (loss divergence、reward drop > threshold)。

Course reminders

- Today
 - Homework 1 due, homework 2 out
 - Sending out \$100 AWS credits + form for GPU quota.
 - Extra Q-learning section (1:30-2:30 pm)

Very important if you want GPUs,
especially for default project

2

图 2: Slide: Q-Learning deployment guardrails 与 monitoring stack

7.3 Evidence Matrix 与操作时间线

7.4 Evidence Matrix 与操作时间线

为每轮训练记录 evidence matrix, 可帮助快速定位 failure root cause。同一矩阵也能映射到 action timeline:

Evidence	Action
Prediction explosion	回滚至低 learning rate checkpoint + 增加 target damping
Reward spike	检查 reward shaping logs, 暂停 training 并审查 newly added shaping term
TD error drift	触发 replay augmentation、增加 PER alpha
Policy collapse (success rate drop)	Enforce early rollback + high-priority fail buffer replay

表 2: Evidence matrix 与对应行动时间线

Actions should be traceable

为每次 triggered action 记录 context (checkpoint id、evidence source、actor responsible), 方便审计和 replay review。

7.5 本章小结

评估矩阵和治理 checklist 保证 Q-Learning 算法不会在部署后被 reward hack 或 value explosion 破坏。

8 工程化视角：Q-Learning workflow

8.1 数据 → 训练 → 部署闭环

工程项目的典型闭环包含：数据收集 → 模型训练 → 评估复核 → Sim rollout + human feedback → 线上监控。

闭环流程关键节点

1. 收集 high-quality transitions, 避免 agent 作弊。2. 用 PER + Double Q + regularization 做训练稳定化。3. 增强评估范式：human-in-the-loop replay、failure case tagging。4. 设定 monitoring guardrail：TD error、reward variance、latency。

8.2 部署监控策略

部署后重点监控：

- ‘TD error’ drift（若上升需重新训练/rollback）
- reward variance（如变小可能代表 behavior collapse）
- ‘episode success rate’ 与 ‘reward per step’，确保性能不退化

8.3 本章小结

工程化部署强调 replay buffer hygiene、TD error 警戒和 telemetry-driven rollback。

9 可视证据与研究时间线

9.1 Slide 3: Q-Learning 模型管线

这一页展示 Q-Learning from data 到 deployment 的 pipeline, 每个环节都需要 guardrail (replay hygiene、target network、threshold monitor)。

Recap: Some Useful Objects

value function $V^\pi(\mathbf{s})$ - future expected rewards starting at $\mathbf{s}$ and following π

Q-function $Q^\pi(\mathbf{s}, \mathbf{a})$ - future expected rewards starting at $\mathbf{s}$, taking $\mathbf{a}$, then following π

$$V^\pi(\mathbf{s}_t) = \sum_{t'=t}^T E_{\pi_\theta} [r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) | \mathbf{s}_t] \quad Q^\pi(\mathbf{s}_t, \mathbf{a}_t) = \sum_{t'=t}^T E_{\pi_\theta} [r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) | \mathbf{s}_t, \mathbf{a}_t]$$

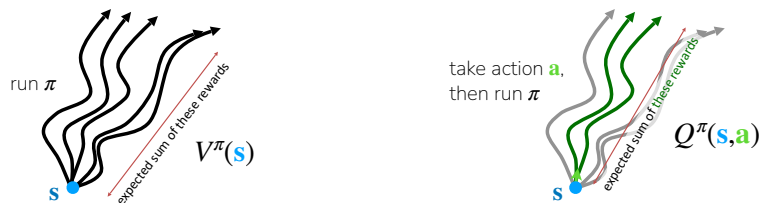


图 3: Slide: Q-Learning pipeline and guardrail checks

9.2 Slide 4: Temporal difference 与 replay dynamics

Slide 4 把 TD error 与 replay sampling 关联起来，强调 ‘high-error’ 采样对 prioritized buffer 的贡献同时也要人工审查。

Recap: Methods

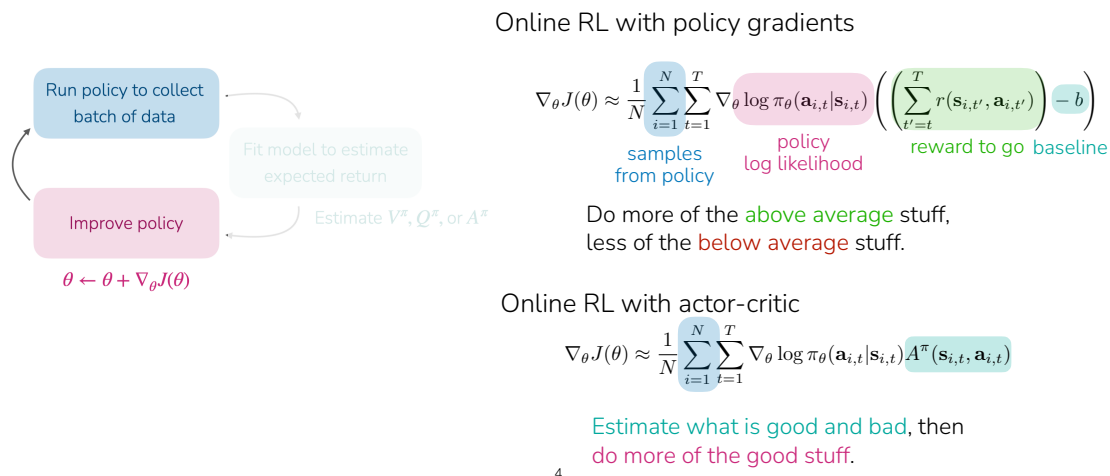
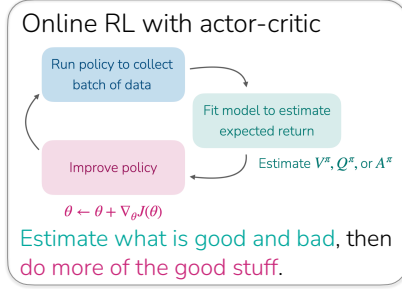


图 4: Slide: TD error monitoring 与 prioritized sampling

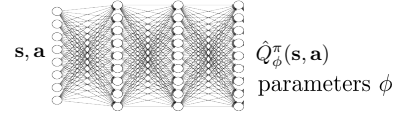
9.3 Slide 5: Deployment 监控

部署时需要对 reward distribution、episode success rate、latency 进行多通道监控，Slide 5 展示了 telemetry stack。

Recap: Off-Policy Policy Evaluation



For off-policy version: can we estimate **Q-function** using past policy data?



For any $(\mathbf{s}, \mathbf{a})$:

$$Q^{\pi_\theta}(\mathbf{s}, \mathbf{a}) = r(\mathbf{s}, \mathbf{a}) + \gamma \mathbb{E}_{\mathbf{s}' \sim p(\cdot | \mathbf{s}, \mathbf{a}), \bar{\mathbf{a}}' \sim \pi_\theta(\cdot | \mathbf{s}')} [Q^{\pi_\theta}(\mathbf{s}', \bar{\mathbf{a}}')]$$

1. Sample $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)$ from the buffer
2. Sample $\bar{\mathbf{a}}'_i \sim \pi_\theta(\cdot | \mathbf{s}'_i)$ from current policy

$$Q^{\pi_\theta}(\mathbf{s}_i, \mathbf{a}_i) \approx \underbrace{r(\mathbf{s}_i, \mathbf{a}_i) + \gamma Q^{\pi_\theta}(\mathbf{s}'_i, \bar{\mathbf{a}}'_i)}_{y_i}$$

$$\mathcal{L}(\phi) = \frac{1}{N} \sum_i \left\| \hat{Q}_\phi^\pi(\mathbf{s}_i, \mathbf{a}_i) - y_i \right\|^2$$

Note: To be accurate, targets y_i need sufficient action coverage

5

图 5: Slide: deployment telemetry and alert stack

9.4 Slide 6: Action timeline and evidence matrix

Slide 6 附上 evidence matrix, 说明每种 failure 都配有对应 action timeline, 正如我们在本文中增加的 table 所示。

The plan for today

Value-based RL methods

1. Q-learning RL method
 - a. Policy iteration
 - b. Bellman optimality equation
 - c. How to collect data for Q-learning methods
2. Q-learning in practice
 - d. Target networks
 - e. Double DQN
 - f. N-step returns

Part of homework 2!

Key learning goals:

- How Q-functions relate to policies
- How to do RL without learning an explicit policy
- How to stabilize Q-learning in practice

6

图 6: Slide: Evidence matrix and action timeline

9.5 Slide 7: Research directions

这页点出 future direction: multi-step Q-Learning, safety evaluation, sim-to-real.

A thought exercise

Recall: Definition of Q-values

$$Q^\pi(\mathbf{s}_t, \mathbf{a}_t) = \sum_{t'=t}^T E_\pi[r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) | \mathbf{s}_t, \mathbf{a}_t]$$

“total reward we get if we take $\mathbf{a}_t$ in $\mathbf{s}_t$...
... and then follow the policy π ”

For some policy π , say you have an accurate estimate $\hat{Q}^\pi(\mathbf{s}, \mathbf{a})$ for all $\mathbf{s}, \mathbf{a}$

Define a new policy $\pi_{\text{new}}(\mathbf{a}_t | \mathbf{s}_t) = \begin{cases} 1 & \text{if } \mathbf{a}_t = \arg \max_{\mathbf{a}} Q_\phi(\mathbf{s}_t, \mathbf{a}) \\ 0 & \text{otherwise} \end{cases}$



Questions: Is the new policy better, worse, or the same as the original policy?
Is it the optimal policy? Why or why not?

7

图 7: Slide: Research directions for safe Q-Learning

9.6 Slide 8: Time/Loss visualization

Slide 8 展示了 reward curve 与 TD error 的同步下降，为收束全文提供视觉线索。

Can we omit policy gradient completely?

$Q^\pi(\mathbf{s}_t, \mathbf{a}_t)$: expected reward from taking $\mathbf{a}_t$ and subsequently following π

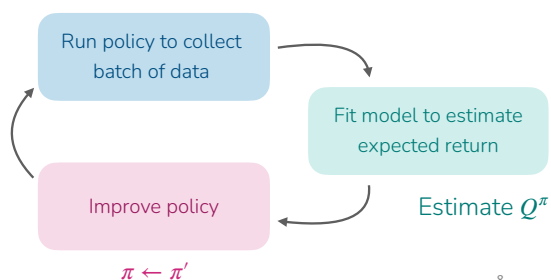
$\arg \max_{\mathbf{a}_t} Q^\pi(\mathbf{s}_t, \mathbf{a}_t)$: best action from $\mathbf{s}_t$, if we then follow π afterwards

at *least* as good as any $\mathbf{a}_t \sim \pi(\mathbf{a}_t|\mathbf{s}_t)$
regardless of what $\pi(\mathbf{a}_t|\mathbf{s}_t)$ is!

forget policies, let's just do this!

$$\pi'(\mathbf{a}_t|\mathbf{s}_t) = \begin{cases} 1 & \text{if } \mathbf{a}_t = \arg \max_{\mathbf{a}} Q^\pi(\mathbf{s}_t, \mathbf{a}) \\ 0 & \text{otherwise} \end{cases}$$

as good as π
(probably better)



Slide adapted from Sergey Levine

8

图 8: Slide: Reward TD error visualization

10 Research Directions & Open Questions

10.1 Scalable replay and prioritization

在高维任务中，传统 replay buffer 不足以覆盖需要探索的新状态。未来研究需要同时解决：

- 如何让 prioritized sampling 兼容 multi-step TD 估计。
- 如何在保证 sampling 多样性的前提下整合 demonstration / human corrections。
- 如何让 buffer adapt to policy shift (如 ‘exponentially weighted recency’)。

Open problem: replay hygiene at scale

在分布式训练中各 worker 写入 buffer 时需确保非重采样、请勿漏采，未来可用 vector clock 来追踪 experience provenance。

10.2 Robust evaluation and failure diagnostics

Q-Learning 的 failure 通常隐蔽：reward hack, return drop, policy collapse。除了 simulation metrics，研究还需解决：

- 提取 ‘evidence matrix’ 并自动关联到 actions。
- 构造 ‘mirrored failure suite’ 以 replay earlier catastrophic transitions。
- 在 production 中保留 ‘checkpoints’ 供 human-in-the-loop 查证。

诊断中最常见的盲点

工程团队往往只关注 reward curve，一旦 reward plateau 误判为 success 就错过了 action drift。建议 combined view (reward + TD error + success rate)。

10.3 Agent orchestration and governance

Q-Learning 进入 agent stack 后，治理就需要跨 levels：

- 让 metric exports 透明 (TD error, max Q)。
- 让 training job 控制 plane 记录 (checkpoint id, data seed, guardrail status)。
- 设定 ‘deployment kill switch’ 以 ‘reward drop > ‘触发 rollback。

Governance checklist

1. 每个 release 带上 evidence matrix + action timeline；2. 训练 pipeline 必须把 ‘reward variance’ 率先视为 safety signal；3. Deployment 过程须支持 ‘fast rollback’。

10.4 本章小结

研究方向集中在 scalable replay、robust evaluation 和 agent-level governance；这些工作将决定 Q-Learning 在更复杂任务中的可信度。最后一页提示了未来 research direction：multi-step q-learning、sim-to-real bridging 和 safety evaluation。

A thought exercise

Recall: *Definition of Q-values*

$$Q^\pi(\mathbf{s}_t, \mathbf{a}_t) = \sum_{t'=t}^T E_\pi [r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) | \mathbf{s}_t, \mathbf{a}_t]$$

“total reward we get if we take $\mathbf{a}_t$ in $\mathbf{s}_t$...
... and then follow the policy π ”

For some policy π , say you have an accurate estimate $\hat{Q}^\pi(\mathbf{s}, \mathbf{a})$ for all $\mathbf{s}, \mathbf{a}$

Define a new policy $\pi_{\text{new}}(\mathbf{a}_t | \mathbf{s}_t) = \begin{cases} 1 & \text{if } \mathbf{a}_t = \arg \max_{\mathbf{a}} Q_\phi(\mathbf{s}_t, \mathbf{a}) \\ 0 & \text{otherwise} \end{cases}$



Questions: Is the new policy better, worse, or the same as the original policy?
Is it the optimal policy? Why or why not?

7

图 9: Slide: Research directions for safe Q-Learning

10.5 Sim-to-real 数据迁移

Q-Learning 部署在真实系统前常需 sim-to-real transfer:

- 在 sim 中加入 observation noise 与 actuator delay 再训练 Q。
- 通过 behavior cloning 将 sim checkpoint 融入 target replay buffer 中。
- 使用 policy distillation 让 smaller net approximate sim net。

迁移监管要点

1. 确认 sim 与 real timeline alignment; 2. 模拟数据必须包含真实 field noise; 3. rollout 全量记录供 human-in-the-loop 审查。

11 总结与延伸

11.1 核心总结表

主题	核心结论	工程提示
Q-Learning 核心	学习最优 Q 函数, 策略由 $\arg \max_a Q(s, a)$ 得出	准备好 replay buffer、target network、regularization
DQN	replay buffer + target network + ϵ -greedy 是可伸缩版 Q-Learning	保持 buffer size、采样多样性与 target network 更新频率
稳定化	Double Q、防止 reward hack 以及 reward shaping	监控 TD error、reward variance; 设置 rollback guardrail
工程部署	评估矩阵 + checklist + telemetry	用 sim rollout + human-in-the-loop replay 验证 checkpoint, 线上 guardrail 监控 Q drift

表 3: Lecture 6 的方法与工程对照

11.2 进一步阅读

11.3 进一步阅读

- Mnih et al., *Playing Atari with Deep Reinforcement Learning* (DQN, 2013)
- Van Hasselt et al., *Deep Reinforcement Learning with Double Q-learning* (Double DQN, 2016)
- Lillicrap et al., *Continuous Control with Deep Reinforcement Learning* (DDPG, 2016)
- Schaul et al., *Prioritized Experience Replay* (ICLR 2016)

- Jaderberg et al., *Population Based Training of Neural Networks* (NeurIPS 2017) —for lifecycle automation

11.4 本章小结

Q-Learning 的价值在于从价值函数推导策略，结合稳定化、数据治理和工程化部署可以让这一经典算法在现代 RL 系统中可靠地运行。

11.5 行动提醒

- 每轮训练后将 evidence matrix 与 order-of-operations 做成文档，方便下一次 rollback。
- 监控 ‘TD error drift’、‘reward variance’、‘episode success rate’，其中任一指标触发 guardrail 后立即 snapshot。
- 安排 quarterly replay audit (human review 20 failure trajectories)。